


# DATA STRUCTURES AND ALGORITHMS

# 23CSE203

# LAB MANUAL

| SUBMITTED BY | |
|---|---|
| NAME | Vaddiparthi Gopal |
| ROLL NO | AV.SC.U4CSE25249 |
| YEAR/SEM/SECTION | 2ND YEAR/SEM-3/CSE-C |
| CREDITS | 05 |

| | |
|---|---|
| SUBMITTED TO | Dr Gayathri Parasa |
| DESIGNATION | Asst. Professor (Sr. Gr) |
| DEPARTMENT | SCHOOL OF COMPUTING |

# INDEX


| S.NO | TITLE | PAGE NO |
|---|---|---|
| 1. | **WEEK 1:**<br>**1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!".<br>**1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate $P^n$(power) where: P= principal growth factor, n= number of years.<br>**1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.<br>**1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.<br>**1(E).** Write a recursive python program to print the first n terms of the Fibonacci series. | 5<br>6<br>7<br>8<br>9 |
| 2. | **WEEK 2:**<br>**2(A).** Write a python program to implement linear search.<br>**2(B).** Write a python program to implement binary search with as an input. | 10<br>11 - 12 |
| 3. | **WEEK-3:**<br>**3(A).** Write a python program to implement bubble sort.<br>**3(B).** Write a python program to implement insertion sort.<br>**3(C).** Write a python program to implement selection sort. | 13<br>14<br>15 |
| 4. | **WEEK-4:**<br>**4(A).** Write a python program to implement quick sort.<br>**4(B).** Write a python program to implement merge sort. | 16<br>17 |
| 5. | **WEEK-5:**<br>**5(A).** Write a python program to implement a Singly Linked List with the below operations.<br>1) Create Linked List<br>2) Insert Beginning<br>3) Insert End<br>4) Insert at Index | 18 - 22 |

| | | |
|---|---|---|
| | 5) Delete by Value<br>6) Delete First Node<br>7) Delete Last Node<br>8) Count number of nodes<br>9) Sort the data of SLL<br>10)Search data in SLL<br>11)Display or Traverse<br>12)Exit | |
| 6. | **WEEK-6:**<br>Write a python program to implement a Double Linked List with the below operations.<br>1) Create Linked List<br>2) Insert Beginning<br>3) Insert End<br>4) Insert at Index<br>5) Delete by Value<br>6) Delete First Node<br>7) Delete Last Node<br>8) Count number of nodes<br>9) Display or Traverse<br>10)Exit | 23 - 27 |
| 7. | **WEEK-7:**<br>**7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array.<br>**7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list. | 28 - 29<br>30 - 31 |
| 8. | **WEEK-8:**<br>**8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.<br>**8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. | 32 - 35<br>36 - 39 |
| 9. | **WEEK-9:**<br>Write a python program to implement a Binary Search Tree (BST) and perform the following operations.<br>1. Insertion<br>2. Deletion | 40 - 44 |

| | | |
|---|---|---|
| | 3. Tree Traversals – in-order, pre-order and post-order<br>4. Count Leaf Nodes<br>5. Count Internal Nodes<br>6. Height of the Tree | |
| 10. | **WEEK-10:**<br>Write a menu driven python program to implement an AVL Tree and perform the following operations.<br>1. Insertion<br>2. Deletion<br>3. Tree Traversals<br>4. Count Leaf Nodes<br>5. Count Internal Nodes<br>6. Height of the Tree | 45 - 49 |
| 11. | **WEEK-11:**<br>Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:<br>1. Insertion of an element into the Heap.<br>2. Deletion of the root element from the Heap.<br>3. Display the array representation of the Heap.<br>4. Exit the program. | 50 - 53 |
| 12. | **WEEK-12:**<br>Write a program to implement the following three collision resolution techniques in a Hash Table:<br>1. Separate Chaining<br>2. Linear Probing<br>3. Quadratic Probing | 54 - 59<br><br>54 - 55<br>56 - 57<br>58 - 59 |

# WEEK – 1

## PROGRAM-1(A):

**AIM:** To write a recursive Python program to display a countdown from n to 1, followed by "Launch!".

## CODE:

```
def countdown(n):
    if n == 0:
        print("Launch!")
        return
    print(n)
    countdown(n - 1)

n = int(input("Enter the starting number: "))
countdown(n)
```

## OUTPUT:

```
Enter countdown number: 10
10
9
8
7
6
5
4
3
2
1
Launch!
```

## PROGRAM-1(B):

**AIM:** To write a recursive Python program to calculate $P^n$ (P = principal growth factor, n = number of years) for compound growth of an investment.

## CODE:

```
def power(P, n):
    if n == 0:
        return 1
    return P * power(P, n - 1)

P = float(input("Enter the principal growth factor (P): "))
n = int(input("Enter the number of years (n): "))
print("P ^ n =", round(power(P, n), 4))
```

## OUTPUT:

```
Enter base (P): 3
Enter exponent (n): 2
Result = 9
```

## PROGRAM-1(C):

**AIM:** To write a recursive Python function to search for a given employee ID in a list.

## CODE:

```
def search_id(ids, key, index):
    if index == len(ids):
        return -1
    if ids[index] == key:
        return index
    return search_id(ids, key, index + 1)

ids = list(map(int, input("Enter employee IDs separated by ',': ").split(',')))
key = int(input("Enter the employee ID to be search: "))
res = search_id(ids, key, 0)
if res != -1:
    print("Employee ID", key, "found at index", res)
else:
    print("Employee ID not found")
```

## OUTPUT:

```
Enter Employee ID to search: 102
Employee ID Found
```

## PROGRAM-1(D):

**AIM:** To write a recursive Python program to calculate the factorial of a given number.

## CODE:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

n = int(input("Enter a number: "))
if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    print("Factorial = ",factorial(n))
```

## OUTPUT:

```
Enter a number: 5
Factorial = 120
```

## PROGRAM-1(E):

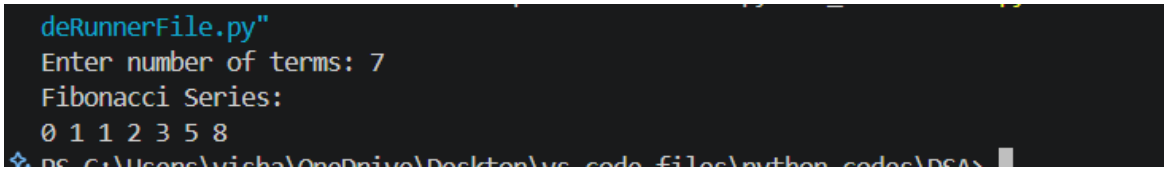
**AIM:** To write a recursive Python program to print the first n terms of the Fibonacci series.

## CODE:

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

n = int(input("Enter number of terms: "))
for i in range(n):
    print(fib(i), end=" ")
```

## OUTPUT:



# WEEK – 2

## PROGRAM-2(A):

**AIM:** To write a Python program to implement linear search.

## CODE:

```
def linear_search(arr,k):
    for i in range(len(arr)):
        if arr[i]==k:
            return i
    return -1
arr=list(map(int,input("Enter the array with separating by space: ").split()))
k=int(input("Enter the element to search for: "))
re=linear_search(arr,k)
if re!=-1:
    print(k,"  is found at index : ", re)
else:
    print("Not Found")
```

## OUTPUT:

```
Enter elements of the array separated by space: 10 20 12 32 42 16
Enter the element to search for: 12
Element found at index: 2
```

## PROGRAM-2(B):

**AIM:** To write a Python program to implement binary search with a sorted array as an input.

## CODE:

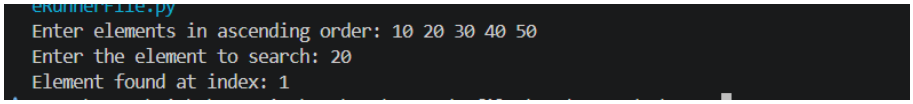
***Binary Search (Iterative) – sorted array***

```
def binary_search(arr,k):
    lo=0
    high=len(arr)-1
    while lo<=high:
        mid=(lo+high)//2
        if arr[mid]==k:
            return mid
        elif arr[mid]>k:
            high=mid-1
        else:
            lo=mid+1
    return -1
arr=list(map(int,input("Enter the array with separating by ',': ").split(',')))
k=int(input("Enter the key to be found: "))
re=binary_search(arr,k)
if re!=-1:
    print(k,"  is found at: ", re)
else:
    print("Not Found")
```

***Binary Search (Recursive) – sorted array***

```
def binary_search(lo,high,arr,k):
    if lo<=high:
        mid=(lo+high)//2
        if arr[mid]==k:
            return mid
        elif arr[mid]>k:
            return binary_search(lo,mid-1,arr,k)
        else:
            return binary_search(mid+1,high,arr,k)
    else:
        return -1
arr=list(map(int,input("Enter the array in ascending order: ").split(',')))
k=int(input("Enter the element to search "))
re=binary_search(0,len(arr)-1,arr,k)
if re!=-1:
    print(k,"  is found at: ", re)
else:
    print("Not Found")
```

## OUTPUT:


Enter elements in ascending order: 10 20 30 40 50
Enter the element to search: 20
Element found at index: 1

# WEEK – 3

## PROGRAM-3(A):

**AIM:** To write a Python program to implement bubble sort.

## CODE:

```
def BubbleSort(arr):
    for i in range(len(arr)-1):
        for j in range(len(arr)-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]

arr=list(map(int,input("Enter Numbers: ").split(',')))
BubbleSort(arr)
for i in arr:
    print(i,end=" ")
```

## OUTPUT:

```
Enter numbers: 10 3 4 20 12 19
Sorted List: [3, 4, 10, 12, 19, 20]
```

## PROGRAM-3(B):

**AIM:** To write a Python program to implement insertion sort.

## CODE:

```
def Insertion_Sort(arr):
    for i in range(1,len(arr)):
        key=arr[i]
        j=i-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j=j-1
        arr[j+1]=key
arr=list(map(int,input("Enter Numbers : ").split()))
Insertion_Sort(arr)
for i in arr:
    print(i,end=" ")
```

## OUTPUT:

```
eRunnerFile.py"
Enter numbers: 8 4 19 12 3
Sorted List: [3, 4, 8, 12, 19]
PS C:\Users\vicha\OneDrive\Desktop\vs code files\python codes\DSA>
```

## PROGRAM-3(C):

**AIM:** To write a Python program to implement selection sort.

## CODE:

```
def Selection_Sort(arr):
    for i in range(len(arr)-1):
        min=i
        for j in range(i+1,len(arr)):
            if arr[j]<arr[min]:
                min=j
        arr[i],arr[min]=arr[min],arr[i]
arr=list(map(int,input("Enter Numbers split(,): ").split(',')))
Selection_Sort(arr)
for i in arr:
    print(i,end=" ")
```

## OUTPUT:

```
 on_sort.py"
 Enter numbers: 20 13 18 2 7
 Sorted List: [2, 7, 13, 18, 20]
```

# WEEK – 4

## PROGRAM-4(A):

**AIM:** To write a Python program to implement quick sort.

## CODE:

```
def quicksort(a, low, high):
    if low < high:
        i = low
        j = high
        pivot = low
        while i < j:
            while i < len(a) and a[i] <= a[pivot]:
                i += 1
            while a[j] > a[pivot]:
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
        a[j], a[pivot] = a[pivot], a[j]
        quicksort(a, low, j - 1)
        quicksort(a, j + 1, high)

a = list(map(int, input("Enter Numbers split(,): ").split(',')))
quicksort(a, 0, len(a) - 1)
print("Sorted array:")
for i in a:
    print(i, end=" ")
```

## OUTPUT:

```
Enter numbers: 1 2 20 12 17 5 3
Sorted List: [1, 2, 5, 3, 12, 20, 17]
PS C:\Users\visha\OneDrive\Desktop\vs code files\python codes\DSA>
```

## PROGRAM-4(B):

**AIM:** To write a Python program to implement merge sort.

## CODE:

```
def merging(arr, low, mid, high):
    result = []
    i = low
    j = mid + 1
    while i <= mid and j <= high:
        if arr[i] < arr[j]:
            result.append(arr[i])
            i += 1
        else:
            result.append(arr[j])
            j += 1
    while i <= mid:
        result.append(arr[i])
        i += 1
    while j <= high:
        result.append(arr[j])
        j += 1
    i = low
    j = 0
    while i <= high:
        arr[i] = result[j]
        i += 1
        j += 1

def merge_sort(arr, low, high):
    if low < high:
        mid = (low + high) // 2
        merge_sort(arr, low, mid)
        merge_sort(arr, mid + 1, high)
        merging(arr, low, mid, high)

arr = list(map(int, input("Enter Numbers split(,): ").split(',')))
n = len(arr)
merge_sort(arr, 0, n - 1)
print("Sorted array:")
for i in arr:
    print(i, end=" ")
```

## OUTPUT:

```
Enter numbers: 8 3 7 12 19 20 22
Sorted List: [3, 7, 8, 12, 19, 20, 22]
```

# WEEK – 5

## PROGRAM-5(A):

**AIM:** To write a Python program to implement a Singly Linked List with the operations: Create, Insert at Beginning, Insert at End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Sort, Search, Display/Traverse and Exit.

## CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of elements: "))
        for i in range(n):
            x = int(input("Enter a value: "))
            new = Node(x)
            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    def insert_index(self, index, data):
        if index < 0:
            print("Invalid Index")
            return
```

```
        if index == 0:
            self.insert_begin(data)
            return
        new = Node(data)
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid Index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid Index")
            return
        new.next = temp.next
        temp.next = new

    def deleteAtBeg(self):
        if self.head is None:
            print("No Data to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted value =", temp.data)

    def deleteAtEnd(self):
        if self.head is None:
            print("No Data to Delete")
        elif self.head.next is None:
            self.head = None
        else:
            temp = self.head
            temp1 = temp
            while temp.next:
                temp1 = temp
                temp = temp.next
            temp1.next = None

    def delete(self, value):
        if self.head is None:
            print("No Data to delete")
        else:
            temp = self.head
            if temp and temp.data == value:
                self.head = temp.next
                print("Value deleted")
                return
            while temp.next and temp.next.data != value:
                temp = temp.next
            if temp.next is None:
                print("Value not present")
            else:
```

```
                temp.next = temp.next.next
                print("Value deleted")

    def count(self):
        if self.head is None:
            print("No Linked List")
        else:
            c = 0
            temp = self.head
            while temp:
                c += 1
                temp = temp.next
            print(f"Number of Nodes = {c}")

    def search(self, k):
        if self.head is None:
            print("Cannot perform search")
            return
        temp = self.head
        while temp:
            if temp.data == k:
                print(f"{k} is found")
                return
            temp = temp.next
        print(f"{k} not found")

    def sort(self):
        if self.head is None:
            print("Cannot perform sorting")
            return
        i = self.head
        while i:
            j = i.next
            while j:
                if i.data > j.data:
                    i.data, j.data = j.data, i.data
                j = j.next
            i = i.next
        print("Sorting Completed")

    def display(self):
        if self.head is None:
            print("No Data")
        else:
            temp = self.head
            while temp:
                print(temp.data, end="-->")
                temp = temp.next
            print("None")

sll = SinglyLinkedList()
```

```python
while True:
    print("\n----- SINGLY LINKED LIST MENU -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count number of nodes")
    print("9. Sort the data of SLL")
    print("10. Search data in SLL")
    print("11. Display or Traverse")
    print("12. Exit")
    n = int(input("Enter your choice: "))
    match n:
        case 1:
            sll.create()
        case 2:
            data = int(input("Enter the value: "))
            sll.insert_begin(data)
        case 3:
            data = int(input("Enter the value: "))
            sll.insert_end(data)
        case 4:
            index = int(input("Enter the index: "))
            data = int(input("Enter the value: "))
            sll.insert_index(index, data)
        case 5:
            value = int(input("Enter the value to delete: "))
            sll.delete(value)
        case 6:
            sll.deleteAtBeg()
        case 7:
            sll.deleteAtEnd()
        case 8:
            sll.count()
        case 9:
            sll.sort()
        case 10:
            k = int(input("Enter your target: "))
            sll.search(k)
        case 11:
            sll.display()
        case 12:
            print("Program Exited")
            break
        case _:
            print("Invalid Choice")
```

# OUTPUT:

```
eRunnerFile.py"
Menu:
1. Create Linked List
2. Display Linked List
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Search for an Element
10. Count Nodes
11. Delete a specific element
12. Sort the linked list
13. Exit
Enter your choice: 1
Enter the number of the nodes: 5
enter the data for node : 1
enter the data for node : 2
enter the data for node : 3
enter the data for node : 4
enter the data for node : 5

Menu:
1. Create Linked List
2. Display Linked List
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Search for an Element
10. Count Nodes
11. Delete a specific element
12. Sort the linked list
13. Exit
Enter your choice: 2
1 -> 2 -> 3 -> 4 -> 5 -> None
```

```
Menu:
1. Create Linked List
2. Display Linked List
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Search for an Element
10. Count Nodes
11. Delete a specific element
12. Sort the linked list
13. Exit
Enter your choice: 6

Menu:
1. Create Linked List
2. Display Linked List
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Search for an Element
10. Count Nodes
11. Delete a specific element
12. Sort the linked list
13. Exit
Enter your choice: 2
2 -> 3 -> 4 -> 5 -> None
```

# WEEK – 6

## PROGRAM-6:

**AIM:** To write a Python program to implement a Doubly Linked List with the operations: Create, Insert at Beginning, Insert at End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Display/Traverse and Exit.

## CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def create(self):
        n = int(input("Enter number of values: "))
        for i in range(n):
            x = int(input("Enter value: "))
            new = Node(x)
            if self.head is None:
                self.head = new
                self.tail = new
            else:
                self.tail.next = new
                new.prev = self.tail
                self.tail = new

    def insert_begin(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            self.tail = new
        else:
            self.head.prev = new
            new.next = self.head
            self.head = new

    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            self.tail = new
        else:
            self.tail.next = new
```

```
            new.prev = self.tail
            self.tail = new

    def insert_index(self, index, data):
        if index < 0:
            print("Invalid Index")
            return
        if index == 0:
            self.insert_begin(data)
            return
        n = self.count()
        if index > n:
            print("Invalid Index")
            return
        if index == n:
            self.insert_end(data)
            print("Insertion Complete")
            return
        # Insert at specific position
        new = Node(data)
        temp = self.head
        for i in range(index - 1):
            temp = temp.next
        new.next = temp.next
        new.prev = temp
        temp.next.prev = new
        temp.next = new
        print("Insertion Completed")

    def delete_Beg(self):
        if self.head is None:
            print("Can't perform")
        elif self.head.next is None:
            self.head = None
            self.tail = None
            print("Value deleted")
        else:
            self.head = self.head.next
            self.head.prev = None
            print("Value deleted")

    def delete_End(self):
        if self.head is None:
            print("Cannot perform")
        elif self.head.next is None:
            self.head = None
            self.tail = None
            print("Value deleted")
        else:
            self.tail = self.tail.prev
            self.tail.next = None
```

```
            print("Value deleted")

    def delete_value(self, value):
        if self.head is None:
            print("Can't perform")
            return
        temp = self.head
        while temp and temp.data != value:
            temp = temp.next
        if temp is None:
            print("Value not found")
        elif temp == self.head:
            self.delete_Beg()
        elif temp == self.tail:
            self.delete_End()
        else:
            temp.prev.next = temp.next
            temp.next.prev = temp.prev
            print("Value deleted")

    def deleteIndex(self, index):
        if self.head is None:
            print("Can't perform")
        elif index < 0 or index >= self.count():
            print("Invalid Index")
        elif index == 0:
            self.delete_Beg()
        elif index == self.count() - 1:
            self.delete_End()
        else:
            temp = self.head
            for i in range(index):
                temp = temp.next
            temp.next.prev = temp.prev
            temp.prev.next = temp.next
            print("Value Deleted")

    def count(self):
        count = 0
        temp = self.head
        while temp:
            count += 1
            temp = temp.next
        return count

    def display_forward(self):
        if self.head is None:
            print("No Data")
            return
        temp = self.head
        while temp:
```

```
            print(temp.data, end="<->")
            temp = temp.next
        print("None")

    def display_Bwd(self):
        if self.head is None:
            print("No Data")
            return
        temp = self.tail
        while temp:
            print(temp.data, end="<->")
            temp = temp.prev
        print("None")

dll = DoublyLinkedList()
while True:
    print("\n----- DOUBLY LINKED LIST MENU -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Delete at Index")
    print("9. Count number of nodes")
    print("10. Display Forward")
    print("11. Display Backward")
    print("12. Exit")
    n = int(input("Enter your choice: "))
    match n:
        case 1:
            dll.create()
        case 2:
            data = int(input("Enter the value: "))
            dll.insert_begin(data)
        case 3:
            data = int(input("Enter the value: "))
            dll.insert_end(data)
        case 4:
            index = int(input("Enter the index: "))
            data = int(input("Enter the value: "))
            dll.insert_index(index, data)
        case 5:
            value = int(input("Enter the value to delete: "))
            dll.delete_value(value)
        case 6:
            dll.delete_Beg()
        case 7:
            dll.delete_End()
        case 8:
```

```
            index = int(input("Enter the index: "))
            dll.deleteIndex(index)
        case 9:
            print("Number of nodes =", dll.count())
        case 10:
            dll.display_forward()
        case 11:
            dll.display_Bwd()
        case 12:
            print("Program Exited")
            break
        case _:
            print("Invalid Choice")
```

## OUTPUT:

```
1.Create  2.Insert beginning  3.Insert end  4.Insert index
5.Delete value  6.Delete index  7.Count  8.Display forward
9.Display backward  10.Exit
Enter choice: 1
Enter number of nodes: 6
Enter data: 1
Enter data: 2
Enter data: 3
Enter data: 4
Enter data: 5
Enter data: 6

1.Create  2.Insert beginning  3.Insert end  4.Insert index
5.Delete value  6.Delete index  7.Count  8.Display forward
9.Display backward  10.Exit
Enter choice: 8
1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <-> None

1.Create  2.Insert beginning  3.Insert end  4.Insert index
5.Delete value  6.Delete index  7.Count  8.Display forward
9.Display backward  10.Exit
Enter choice: 5
Enter value: 3

1.Create  2.Insert beginning  3.Insert end  4.Insert index
5.Delete value  6.Delete index  7.Count  8.Display forward
9.Display backward  10.Exit
Enter choice:
```

# WEEK – 7

## PROGRAM-7(A):

**AIM:** To write a Python program to implement stack operations such as push, pop, peek and display using an array.

## CODE:

```
class StackEx:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "Popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is Empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            for i in range(self.top, -1, -1)
                print(self.stack[i])

size = int(input("Enter the size of stack: "))
s = StackEx(size)
while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
```

```
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program Exited")
        break
    else:
        print("Invalid Choice")
```

## OUTPUT:

```
1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 2
2 pushed into stack

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 2
2 pushed into stack

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 4
2
2

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 2
2 popped from stack

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 4
2

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice:
```

## PROGRAM-7(B):

**AIM:** To write a Python program to implement stack operations such as push, pop, peek and display using a linked list.

## CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "pushed into stack")

    def pop(self):
        if self.top is None:
            print("Stack empty")
        else:
            item = self.top
            self.top = self.top.next
            print(item.data, "Popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is Empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next

s = Stack()
while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
```

```
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program Exited")
        break
    else:
        print("Invalid Choice")
```

## OUTPUT:

```
1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 2
2 pushed into stack

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 2
2 pushed into stack

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 4
2
2

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 2
2 popped from stack

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 4
2

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice:
```

# WEEK – 8

## PROGRAM-8(A):

**AIM:** To write a Python program to implement a Queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach, using array and linked list data structures.

## CODE:

***Queue using array***

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the Queue")

    def dequeue(self):
        if self.rear == -1:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the Queue")
            # Resets Queue when it becomes empty
            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def display(self):
        if self.front == -1:
            print("Queue is Empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])

    def peek(self):
        if self.front == -1:
```

```
            print("Queue is Empty")
        else:
            print("Front element is:", self.queue[self.front])

size = int(input("Enter the size of queue: "))
q = Queue(size)
while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter the choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program Exited")
        break
    else:
        print("Invalid Choice")
```

***Queue using linked list***

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, x):
        new = Node(x)
        if self.rear is None:
            self.front = new
            self.rear = new
        else:
            self.rear.next = new
            self.rear = new
        print(f"{x} inserted into the Queue")

    def dequeue(self):
```

```
        if self.front is None:
            print("Queue Underflow")
        else:
            x = self.front.data
            self.front = self.front.next
            if self.front is None:
                self.rear = None
            print(f"{x} deleted from the Queue")

    def peek(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front element is:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("The elements of the queue are:")
            temp = self.front
            while temp:
                print(temp.data)
                temp = temp.next

q = Queue()
while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter the choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program Exited")
        break
    else:
        print("Invalid Choice")
```

## OUTPUT:


Enter queue size: 4
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter item: 1
1 inserted into queue
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter item: 2
2 inserted into queue
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter item: 3
3 inserted into queue
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 4
[1, 2, 3]
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 2
1 deleted from queue
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice: 4
[2, 3]
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Enter choice:

## PROGRAM-8(B):

**AIM:** To write a Python program to implement a Circular Queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach, using array and linked list data structures.

## CODE:

***Circular Queue using array***

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")


    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        # Only one element is present
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        # More than one element
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    # Peek operation
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
```

```
            print("Front element is:", self.queue[self.front])

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size

# Create Circular Queue
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)
while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

***Circular Queue using linked list***

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None
```

```
    def enqueue(self, item):
        new = Node(item)
        if self.front is None:
            self.front = new
            self.rear = new
            new.next = new
        else:
            self.rear.next = new
            new.next = self.front
            self.rear = new
        print(item, "inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        elif self.front == self.rear:
            item = self.front.data
            self.front = None
            self.rear = None
            print(item, "deleted from the queue")
        else:
            item = self.front.data
            self.front = self.front.next
            self.rear.next = self.front
            print(item, "deleted from the queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element is:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            temp = self.front
            while True:
                print(temp.data)
                temp = temp.next
                if temp == self.front:
                    break

q = CircularQueue()
while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
```

```
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

## OUTPUT:

```
1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 2
2 inserted into the queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 3
3 inserted into the queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 4
4 inserted into the queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
2 -> 3 -> 4 -> (front)

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 2
2 deleted from the queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
3 -> 4 -> (front)

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice:
```

# WEEK – 9

## PROGRAM-9:

**AIM:** To write a Python program to implement a Binary Search Tree (BST) and perform Insertion, Deletion, Tree Traversals (in-order, pre-order, post-order), Count Leaf Nodes, Count Internal Nodes and find the Height of the tree.

## CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
```

```
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
```

```
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)


def level_order(root):
    h = height(root)
    for level in range(h + 1):
        print_level(root, level)


# ======================================================
# MAIN PROGRAM
# ======================================================

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
```

```
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
```

```
            if node is None:
                print("Element", data, "is not found.")
            else:
                print("Element", data, "is found.")
        elif choice == 8:
            data = int(input("Enter the data to be deleted: "))
            root = delete(root, data)
        elif choice == 9:
            print("Number of nodes:", count_nodes(root))
        elif choice == 10:
            print("Number of leaf nodes:", count_leaves(root))
        elif choice == 11:
            print("Number of internal nodes:", count_internal(root))
        elif choice == 12:
            print("Height of BST:", height(root))
        elif choice == 13:
            print("Level Order Traversal:", end=" ")
            if root is None:
                print("Tree is empty.")
            else:
                level_order(root)
                print()
        elif choice == 14:
            print("Program terminated.")
            break
        else:
            print("Invalid choice. Please try again.")
```

## OUTPUT:

```
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 12
Element inserted successfully.


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice:
```

# WEEK – 10

## PROGRAM-10:

**AIM:** To write a menu driven Python program to implement an AVL Tree and perform Insertion, Deletion, Tree Traversals, Count Leaf Nodes, Count Internal Nodes and find the Height of the tree.

## CODE:

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0

#AVL TREE Program
def height(root):
    if root is None:
        return -1
    else:
        return root.h

# LL Rotation
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# RR Rotation
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# LR Rotation
def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)

# RL Rotation
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)
```

```python
# Inorder Traversal
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder Traversal
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


# Postorder Traversal
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# Count Leaf Nodes
def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

# Count Internal Nodes
def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return 1 + count_internal(root.left) + count_internal(root.right)

# Insert a node into AVL Tree
def insert(root, x):
    if root is None:
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
```

```
                root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")
    root.h = max(height(root.left),height(root.right)) + 1
    return root


# Delete a node from AVL Tree
def delete(root, x):
    # Element not found
    if root is None:
        return None

    # Search in left subtree
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    # Search in right subtree
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)

    # When target found
    else:
        if root.left is None and root.right is None:
            root = None
        elif root.right is None:
            root = root.left
        elif root.left is None:
            root = root.right
        else:
            successor = root.right   # Find inorder successor
            while successor.left is not None:
                successor = successor.left
```

```
            root.data = successor.data   #Copy successor data
            root.right = delete(root.right, successor.data)   # Delete the
inorder successor

        root.h = max(height(root.left), height(root.right)) + 1
    return root

# ------------------------------
# Main Program
# ------------------------------

root = None

while True:

    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
    print("6. COUNT LEAF NODES")
    print("7. COUNT INTERNAL NODES")
    print("8. HEIGHT OF TREE")
    print("9. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
    elif choice == 2:
        print("Preorder Traversal:")
        preorder(root)
        print()
    elif choice == 3:
        print("Inorder Traversal:")
        inorder(root)
        print()
    elif choice == 4:
        print("Postorder Traversal:")
        postorder(root)
        print()
    elif choice == 5:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 6:
        print("Number of Leaf Nodes:", count_leaves(root))
    elif choice == 7:
        print("Number of Internal Nodes:", count_internal(root))
    elif choice == 8:
        print("Height of AVL Tree:", height(root))
```

```
    elif choice == 9:
        print("Program terminated.")
        break
    else:
        print("Invalid choice!")
```

## OUTPUT:

```
***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 1
Enter the data to be inserted: 12

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 1
Enter the data to be inserted: 12
Duplicates not allowed

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 2
Preorder Traversal:
12

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
```

# WEEK – 11

## PROGRAM-11:

**AIM:** To write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform Insertion, Deletion of the root element and Display of the array representation of the heap.

## CODE:

***Max Heap***

```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()

    n = len(heap)
    i = 0
    # Move the root downward
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and heap[left] > heap[largest]:
            largest = left
        if right < n and heap[right] > heap[largest]:
            largest = right
        if largest != i:
            heap[i], heap[largest] = heap[largest], heap[i]
            i = largest
        else:
            break

def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
```

```
        print("Heap elements:", heap)

#Main Program
heap = []
while True:
    print("\n--- MAX HEAP MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value)
    elif choice == 2:
        delete(heap)
    elif choice == 3:
        display(heap)
    elif choice == 4:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

***Min Heap***

```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] < heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()

    n = len(heap)
    i = 0
    # Move the root downward
    while True:
        smallest = i
        left = 2 * i + 1
        right = 2 * i + 2
```

```
        if left < n and heap[left] < heap[smallest]:
            smallest = left
        if right < n and heap[right] < heap[smallest]:
            smallest = right
        if smallest != i:
            heap[i], heap[smallest] = heap[smallest], heap[i]
            i = smallest
        else:
            break

def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)

#Main Program
heap = []
while True:
    print("\n--- MIN HEAP MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value)
    elif choice == 2:
        delete(heap)
    elif choice == 3:
        display(heap)
    elif choice == 4:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

## OUTPUT:

```
--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 12

--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 11

--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Heap elements: [12, 11]

--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice:
```

# WEEK – 12

## PROGRAM-12(A):

**AIM:** To write a Python program to implement a Hash Table using Separate Chaining for collision resolution.

## CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

SIZE = 10
ht = [None] * SIZE


def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode


def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
```

```
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

## OUTPUT:

```
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 12

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 11

1. Insert
2. Display
3. Exit
Enter your choice: 2

Hash table values:
0: None
1: 11 -> None
2: 12 -> None
3: None
4: None
5: None
6: None
7: None
8: None
9: None

1. Insert
2. Display
3. Exit
Enter your choice:
```

## PROGRAM-12(B):

**AIM:** To write a Python program to implement a Hash Table using Linear Probing for collision resolution.

## CODE:

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")

def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            print("Element found at index", index)
            return
        if a[index] == -1:
            break
    print("Element not found")


def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Delete")
    print("4. Search")
    print("5. Exit")
```

```
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("Values are:")
        displayValue()
    elif ch == 3:
        value = int(input("Enter value to delete: "))
        deleteValue(value)
    elif ch == 4:
        value = int(input("Enter value to search: "))
        search(value)
    elif ch == 5:
        repeat = False
    else:
        print("Invalid choice")
```

## OUTPUT:

```
1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 12
Element inserted

1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 11
Element inserted

1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 2
Values are:
Index   Value
0       -1
1       11
2       12
3       -1
4       -1
5       -1
6       -1
7       -1
8       -1
9       -1
```

## PROGRAM-12(C):

**AIM:** To write a Python program to implement a Hash Table using Quadratic Probing for collision resolution.

## CODE:

```
SIZE = 10
a = [-1] * SIZE

def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE
    if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return
    print("No empty slot found in the probe sequence")


def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

## OUTPUT:


1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 12
Element inserted at index 3
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 11
Element inserted at index 2
1. Insert
2. Display
3. Exit
Enter your choice: 2
Values are:
-1
-1
11
12
-1
-1
-1
-1
-1
-1
1. Insert
2. Display
3. Exit
Enter your choice: